SmartLibrary

Console-Based Library Management System

Course: Data Structures

Language: Java 17+

Type: Console Application

Date: May 2026

Table of Contents

1. Introduction

SmartLibrary is a Java-based console application that simulates a small-scale library management system. It supports two user roles — **Admin** and **Student** — each with a dedicated set of operations. The application was designed as a practical exercise in applying fundamental data structures to solve a real-world problem: efficiently managing a book catalogue and tracking student borrowing activity.

The system stores its book catalogue in a **Binary Search Tree (BST)** keyed by ISBN, enabling efficient ordered retrieval, search, and modification. Each student's borrowing history is maintained in a **linked-list stack**, which naturally presents the most recent activity first. All data is persisted to local text files, allowing the application to restore its state across sessions without requiring a database.

2. Problem Statement

Traditional manual library management involves significant overhead in tracking book availability, managing borrow and return records, and maintaining an organized catalogue. Small libraries and academic institutions often lack access to commercial library management software, leading to several challenges:

1. **Inefficient Book Lookup:** Searching for a specific book in an unorganized collection requires scanning every record, resulting in O(n) lookup time.
2. **No Borrowing History:** Without systematic tracking, it is difficult to determine which books a student has borrowed, when they were borrowed, and whether they have been returned.
3. **Lack of Role Separation:** Library staff and students often need different levels of access — staff should manage the catalogue, while students should only borrow and return books.
4. **Data Loss Between Sessions:** Without persistence, all catalogue and borrowing data is lost when the application terminates.
5. **No Sorted Display:** Users benefit from seeing books in a consistent, sorted order, but maintaining sort order manually is error-prone.

The SmartLibrary project addresses these problems by implementing a role-based library management system that uses purpose-built data structures to deliver efficient search, insertion, and deletion of book records, along with chronological tracking of every borrowing transaction.

3. Objectives

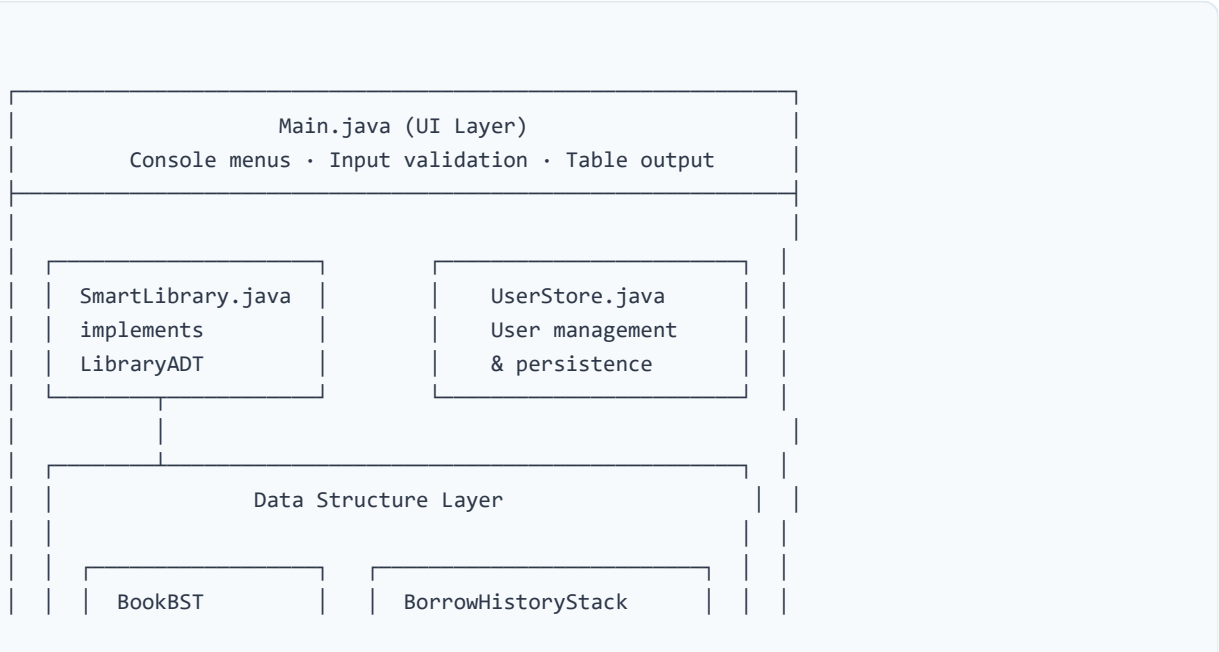
1. Design and implement a **Binary Search Tree** to store and manage book records, enabling O(log n) average-case search, insertion, and deletion by ISBN.
2. Design and implement a **linked-list stack** for each student to record borrowing history in last-in-first-out (LIFO) order.
3. Implement a **role-based access control** system separating Admin and Student operations.
4. Implement **file-based data persistence** so that all catalogue, loan, and user data survives application restarts.
5. Provide a **clean console-based user interface** with input validation, formatted table output, and clear error messages.
6. Apply **abstraction** through a Java interface (`LibraryADT`) to decouple the service layer from the UI.

4. System Design & Approach

4.1 Architectural Overview

The application follows a **layered architecture** with clear separation of concerns:

LAYER	COMPONENT(S)	RESPONSIBILITY
Presentation	`Main.java`	Console menus, user input/output, table formatting
Service	`SmartLibrary.java` `UserStore.java`	Business logic, data validation, persistence orchestration
ADT/Interface	`LibraryADT.java`	Contract for library operations (abstraction layer)
Data Structures	`BookBST.java` `BorrowHistoryStack.java`	Custom BST and Stack implementations
Models	`Book.java`, `LoanRecord.java`, `User.java`	Data carriers (POJOs)
Persistence	`smart_library_data.txt` `user_info.txt`	Pipe-delimited flat-file storage



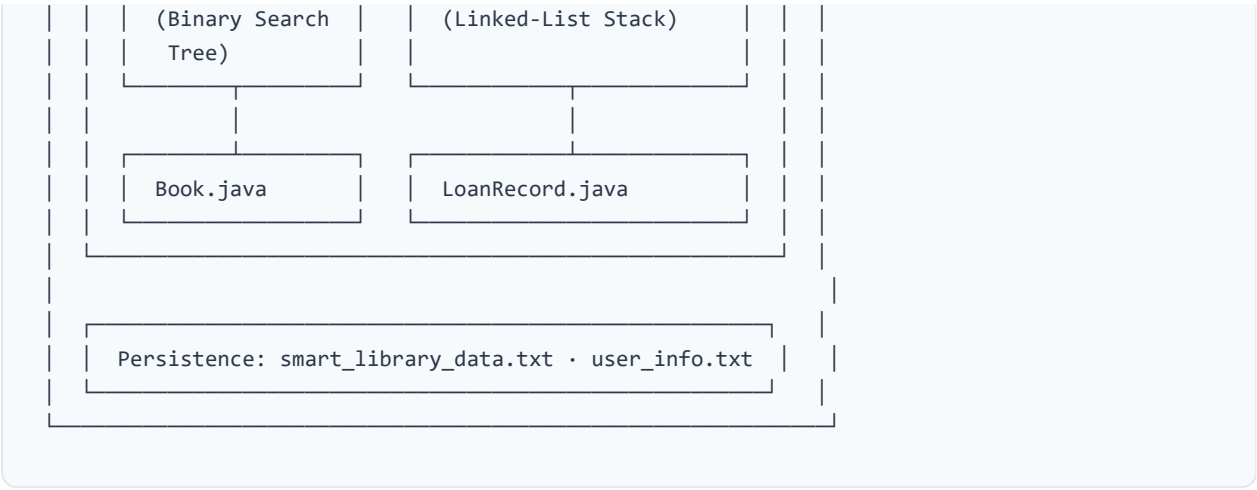


Figure 1: System Architecture Diagram

4.2 Design Decisions

Why a Binary Search Tree for the Catalogue?

A BST was chosen over a simple list or array because the library needs to frequently search for books by ISBN, add new books, and remove books. A BST provides O(log n) average-case performance for all three operations, compared to O(n) for a linear scan. Additionally, an in-order traversal of the BST naturally produces a sorted list of books, which is required for the "View All Available Books" feature.

Why a Stack for Borrowing History?

A stack (LIFO) was chosen for borrowing history because users most naturally want to see their *most recent* activity first. Each new borrow or return event is pushed onto the stack, and when the history is displayed, it reads from top to bottom — newest to oldest. This avoids the need for any explicit sorting step.

Why Flat-File Persistence?

The project uses pipe-delimited text files rather than a database to keep the application simple and dependency-free. The custom escape-character parser ensures that special characters in book titles and author names do not corrupt the data format.

Why an Interface (LibraryADT)?

The `LibraryADT` interface abstracts the library operations so the UI layer (`Main.java`) depends only on the contract, not on the implementation. This allows the underlying data structures to be changed without modifying the UI code.

5. Data Structures Used

5.1 Binary Search Tree — `BookBST`

The `BookBST` class implements a standard binary search tree where each node contains a `Book` object. The tree is keyed by the book's ISBN number (a `long` value).

Node Structure

```
private static class Node {
    private Book book;    // The book data stored at this node
    private Node left;    // Left child (smaller ISBN)
    private Node right;   // Right child (larger ISBN)
}
```

Operations & Complexity

OPERATION	METHOD SIGNATURE	AVERAGE CASE	WORST CASE
Insert	`insert(Book book)`	O(log n)	O(n)
Search	`search(long isbn)`	O(log n)	O(n)
Delete	`deleteByIsbn(long isbn)`	O(log n)	O(n)
In-order traversal	`toList()`	O(n)	O(n)

> **Deletion Strategy:** When a node with two children is deleted, it is replaced by its *in-order successor* — the smallest node in its right subtree. This preserves the BST ordering invariant.

BST Traversal Diagram

```
    [9780134093413]    [9780136042594]   [9780321573513]
    Computer           AI: A Modern       Algorithms
    Networking         Approach
```

Figure 2: Example BST structure (keyed by ISBN, in-order traversal yields sorted output)

5.2 Linked-List Stack — `BorrowHistoryStack`

Each student's borrowing history is maintained in a singly-linked-list stack. The stack uses a custom `Node` class with an immutable reference to a `LoanRecord` and a pointer to the next node.

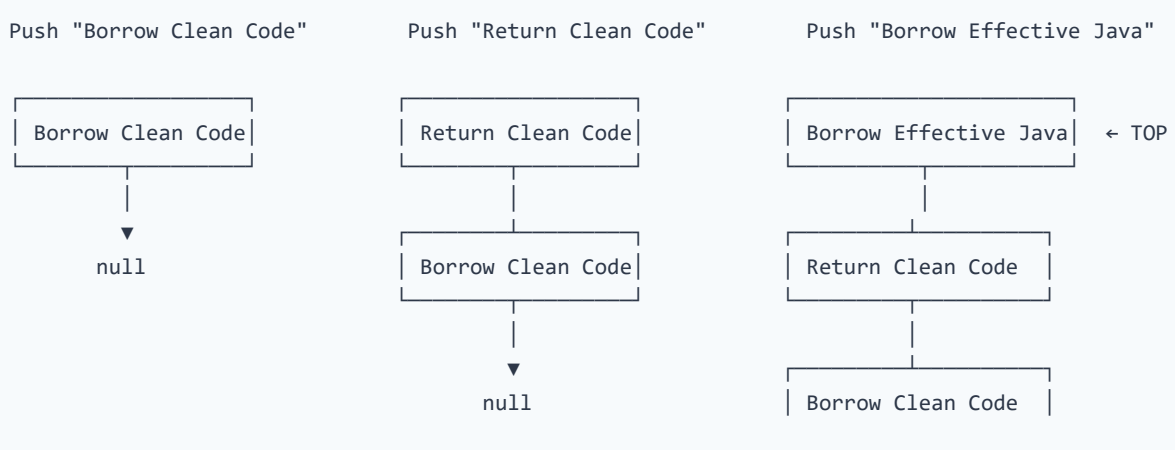
Node Structure

```
private static class Node {
    private final LoanRecord loanRecord;  // Loan data
    private final Node next;              // Next node down the stack
}
```

Operations & Complexity

OPERATION	METHOD SIGNATURE	TIME COMPLEXITY
Push	`push(LoanRecord loanRecord)`	O(1)
Is Empty	`isEmpty()`	O(1)
Find Active Loan	`findActiveLoanByIsbn(long isbn)`	O(n)
Convert to List	`toList()`	O(n)

Stack Behaviour Diagram



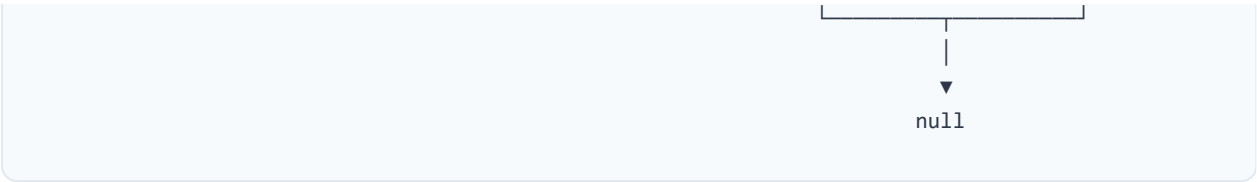


Figure 3: Stack growth — most recent activity is always on top

5.3 LinkedHashMap — Student-to-Stack Mapping

A `LinkedHashMap<String, BorrowHistoryStack>` maps each student ID to their personal history stack. This provides:

- **O(1) lookup** to retrieve a student's history stack by ID
- **Insertion-order iteration** for consistent, reproducible file output
- **Lazy initialization** via `computeIfAbsent()` — a stack is created only when a student first borrows a book

5.4 Summary of Data Structures

DATA STRUCTURE	CLASS	PURPOSE	KEY ADVANTAGE
Binary Search Tree	`BookBST`	Book catalogue storage	O(log n) search/insert/delete; sorted traversal
Linked-List Stack	`BorrowHistoryStack`	Per-student loan history	O(1) push; natural LIFO ordering
LinkedHashMap	(JDK built-in)	Student → Stack mapping	O(1) lookup; insertion-order iteration
ArrayList	(JDK built-in)	Temporary lists, file I/O	Dynamic sizing; indexed access

6. Implementation Details

6.1 File Descriptions

FILE	LINES	ROLE
`Main.java`	525	Console UI — entry point, menus, input reading, formatted table output
`SmartLibrary.java`	424	Service layer — implements `LibraryADT`, manages BST and stacks, handles file I/O
`LibraryADT.java`	53	Interface — defines 9 core library operations
`BookBST.java`	162	BST — insert, search, delete, in-order traversal
`BorrowHistoryStack.java`	71	Stack — push, isEmpty, findActiveLoan, toList
`Book.java`	37	Model — immutable book record (ISBN, title, author)
`LoanRecord.java`	67	Model — borrowing transaction (student, book, dates, status)
`User.java`	29	Model — registered user (ID, name, role)
`UserStore.java`	215	User management — registration, lookup, persistence

Total: ~1,583 lines of Java code across 9 source files.

6.2 Key Algorithms

BST Insertion (Recursive)

New books are inserted by recursively traversing the tree: if the ISBN is less than the current node, go left; if greater, go right; if a `null` child is reached, insert the new node. Duplicate ISBNs are rejected by returning `false`.

BST Deletion (In-order Successor)

When deleting a node with two children, the algorithm finds the in-order successor (the smallest node in the right subtree), copies its book data into the current node, and then recursively deletes the

successor. This maintains the BST invariant without restructuring the entire tree.

Borrowing Workflow

1. Search the BST for the requested ISBN.
2. If found, remove the book from the BST (it is no longer available).
3. Create a `LoanRecord` with today's date and a 14-day due date.
4. Push the loan record onto the student's history stack.
5. Save all data to file.

Return Workflow

1. Look up the student's history stack.
2. Search the stack for an active (unreturned) loan matching the ISBN.
3. If found, mark the loan as returned.
4. Re-insert the book into the BST (it is available again).
5. Save all data to file.

6.3 Data Persistence Format

Data is stored in pipe-delimited text files with escape character support:

```
# smart_library_data.txt
[CATALOGUE]
9780132350884|Clean Code|Robert C. Martin
9780134093413|Computer Networking|James Kurose, Keith Ross

[LOANS]
S001|9780131103627|The C Programming Language|Brian W. Kernighan, Dennis M. Ritchie|2026-05-

# user_info.txt
[USERS]
A001|Default Admin|ADMIN
S001|Default Student|STUDENT
```

> **Escape Handling:** The custom `splitLine()` parser handles backslash-escaped pipe characters (`\|`) and escaped backslashes (`\\`), ensuring that special characters in book titles and author names do not break the data format.

7. User Interface & Workflow

7.1 Application Flow

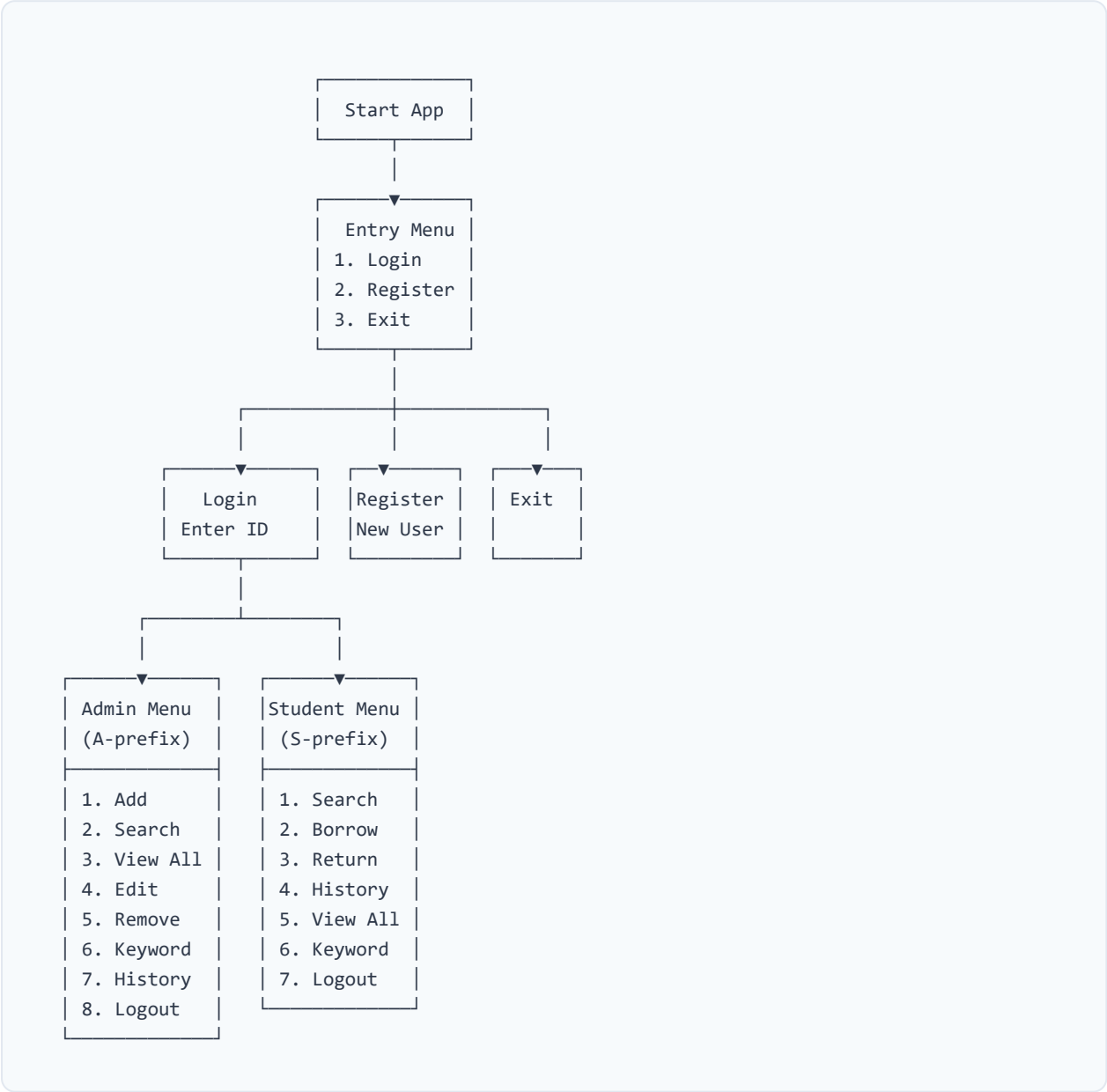


Figure 4: Application Navigation Flow

7.2 Console Output Formatting

The UI layer dynamically calculates column widths based on the actual data content, ensuring that tables are properly aligned regardless of title or author length. This is implemented in the

`printBooks()` and `printLoanRecords()` methods using `String.format()` with computed width specifiers.

7.3 Input Validation

VALIDATION	METHOD	BEHAVIOUR
Empty text fields	`readRequiredText()`	Returns `null` and prints error
Invalid ISBN format	`readPositiveIsbn()`	Rejects non-numeric or negative values
Invalid menu choice	`readMenuChoice()`	Returns -1, triggering the default branch
Invalid user ID format	`isValidUserId()`	Must start with 'A' or 'S'
Duplicate user ID	`registerUser()`	Checks map before insertion
Duplicate ISBN	`BookBST.insert()`	Returns `false` for existing ISBNs

8. Data Persistence

8.1 Persistence Strategy

The application uses a **write-on-change** strategy: the entire data file is rewritten after every state-changing operation (add, edit, remove, borrow, return, register). While this is less efficient than incremental updates, it ensures data consistency and simplifies the implementation for a small-scale application.

8.2 Load Process

1. Check if the data file exists. If not, start with an empty catalogue.
2. Read all lines and parse section headers (`[CATALOGUE]`, `[LOANS]`).
3. Parse book records into `Book` objects and insert into the BST.
4. Parse loan records into `LoanRecord` objects, grouped by student ID.
5. Push loan records into per-student stacks in reverse order to preserve the original stack ordering.

8.3 Legacy Data Migration

The system supports an older `[BORROW_HISTORY]` section format. Records found under this header are automatically migrated to the modern `[LOANS]` format under a special `LEGACY_STUDENT` ID. This ensures backward compatibility when upgrading from older versions.

9. Testing

9.1 Testing Approach

Testing was conducted through **manual functional testing** of all user-facing operations. Each test case was designed to verify both the expected (happy path) and error (edge case) behaviour of the system.

9.2 Functional Test Cases

#	TEST CASE	STEPS	EXPECTED RESULT	STATUS
1	Add a new book	Login as admin → Add Book → Enter valid ISBN, title, author	"Book added successfully." Book appears in catalogue.	✅ Pass
2	Add duplicate ISBN	Add a book with an ISBN already in the catalogue	"Book could not be added." Catalogue unchanged.	✅ Pass
3	Search book by ISBN	Enter a valid ISBN present in the catalogue	Book details displayed in a formatted table.	✅ Pass
4	Search non-existent ISBN	Enter an ISBN not in the catalogue	"No available book found." message displayed.	✅ Pass
5	View all available books	Select "View All Available Books" from menu	All books displayed sorted by ISBN in formatted table.	✅ Pass
6	Edit book details	Login as admin → Edit → Enter ISBN → Enter new title/author	"Book details updated successfully." Updated values appear.	✅ Pass
7	Remove a book	Login as admin → Remove → Enter ISBN	"Book removed permanently." Book no longer in catalogue.	✅ Pass
8	Borrow a book	Login as student → Borrow → Enter available ISBN	"Borrowed successfully." Book removed from available list.	✅ Pass

#	TEST CASE	STEPS	EXPECTED RESULT	STATUS
9	Borrow unavailable book	Try to borrow a book already checked out	"Borrow failed." message displayed.	✅ Pass
10	Return a book	Login as student → Return → Enter borrowed ISBN	"Returned successfully." Book restored to catalogue.	✅ Pass
11	Return a non-borrowed book	Try to return a book not in the student's active loans	"Return failed." message displayed.	✅ Pass
12	View borrowing history	Login as student → View My History	Loan records displayed most-recent-first.	✅ Pass
13	Search by title keyword	Enter partial title (e.g., "Java")	All matching available books displayed.	✅ Pass
14	Search by author keyword	Enter partial author name	All matching available books displayed.	✅ Pass
15	Data persistence	Add books → Exit → Restart application	Previously added books and loans are restored.	✅ Pass

9.3 Edge Case & Validation Tests

#	SCENARIO	INPUT	EXPECTED BEHAVIOUR	STATUS
1	Empty ISBN field	(blank)	"Invalid ISBN. Please enter numbers only."	✅ Pass
2	Negative ISBN	-12345	"ISBN must be a positive number."	✅ Pass
3	Non-numeric ISBN	abc	"Invalid ISBN. Please enter numbers only."	✅ Pass
4	Empty title/author	(blank)	"This field cannot be empty."	✅ Pass
5	Invalid menu choice	99 or "abc"	"Invalid choice. Please select a number from 1 to N."	✅ Pass
6	Register duplicate user ID	A001 (already exists)	"Registration failed. The user ID may already exist."	✅ Pass

#	SCENARIO	INPUT	EXPECTED BEHAVIOUR	STATUS
7	Login with unregistered ID	S999	"User ID not found. Please register first."	✅ Pass
8	Invalid user ID prefix	X001	"Registration failed. Admin IDs must start with A..."	✅ Pass
9	Edit borrowed book	ISBN of a checked-out book	"Edit failed. Only available catalogue books can be edited."	✅ Pass
10	Corrupt data file	Malformed lines in data file	Invalid lines silently skipped; valid data loaded.	✅ Pass
11	Missing data file	Delete data files → start app	Default users created; empty catalogue initialized.	✅ Pass
12	Pipe character in book title	Title containing "|"	Character properly escaped; data saved/loaded correctly.	✅ Pass

10. Results & Discussion

10.1 Achievements

- All 9 operations defined in `LibraryADT` are fully implemented and functional.
- The BST successfully maintains sorted order and provides efficient search by ISBN.
- The stack-based history correctly displays borrowing activity in most-recent-first order.
- Data persistence ensures no data loss between sessions.
- Input validation handles all common error scenarios gracefully.
- The layered architecture cleanly separates UI, business logic, and data structure concerns.

10.2 Limitations

- **Unbalanced BST:** The BST is not self-balancing (e.g., AVL or Red-Black). If books are inserted in sorted ISBN order, the tree degenerates to O(n) performance. For a small library catalogue, this is acceptable.
- **Full-file rewrite:** Every state change rewrites the entire data file. For large datasets, an incremental or database-backed approach would be more efficient.
- **No authentication:** Users log in by ID only; there are no passwords or security measures.
- **Single-copy books:** Each ISBN can appear only once in the catalogue. Real libraries may have multiple copies of the same book.
- **Console-only:** The application has no graphical user interface.

10.3 Possible Enhancements

- Implement an AVL or Red-Black tree for guaranteed O(log n) performance.
- Add password-based authentication for improved security.
- Support multiple copies of the same book (copy counting).
- Migrate to a database (e.g., SQLite) for better scalability.
- Build a graphical or web-based user interface.
- Add overdue notifications and fine calculations.

11. Conclusion

The SmartLibrary project successfully demonstrates how fundamental data structures can be applied to build a practical, working application. The **Binary Search Tree** provides efficient ordered storage and retrieval of book records, while the **linked-list stack** naturally organizes borrowing history in chronological order. The use of a Java interface (`LibraryADT`) to abstract the library operations promotes clean separation between the UI and business logic layers.

Through this project, the following data structure concepts were applied in practice:

- **Binary Search Tree** — insertion, search, deletion (with in-order successor), and in-order traversal
- **Stack (LIFO)** — push, linked-list traversal, and conversion to list
- **Hash Map** — constant-time key-based lookup for student history mapping
- **Abstract Data Type** — interface-based design for decoupling implementation from contract

The project meets all stated objectives: it provides a role-based library management system with efficient book catalogue operations, chronological borrowing history, persistent data storage, and a user-friendly console interface with comprehensive input validation.

12. References

1. Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009). *Introduction to Algorithms*. 3rd ed. MIT Press.
2. Sedgewick, R. and Wayne, K. (2011). *Algorithms*. 4th ed. Addison-Wesley.
3. Bloch, J. (2018). *Effective Java*. 3rd ed. Addison-Wesley.
4. Oracle Corporation. *Java SE 17 API Documentation*. Available at: `https://docs.oracle.com/en/java/javase/17/docs/api/`

13. Appendices

Appendix A: LibraryADT Interface

```
public interface LibraryADT {
    boolean addBook(long isbn, String title, String author);
    Book searchBook(long isbn);
    LoanRecord borrowBook(String studentId, long isbn);
    LoanRecord returnBook(String studentId, long isbn);
    List<LoanRecord> viewBorrowHistory(String studentId);
    List<Book> viewAvailableBooks();
    boolean editBook(long isbn, String title, String author);
    Book removeBook(long isbn);
    List<Book> searchByTitleOrAuthor(String keyword);
}
```

Appendix B: BookBST Core Methods

```
// Insert — O(log n) average
public boolean insert(Book book) {
    if (root == null) {
        root = new Node(book);
        return true;
    }
    return insertRecursive(root, book);
}

// Search — O(log n) average
public Book search(long isbn) {
    return searchRecursive(root, isbn);
}

// Delete — In-order successor strategy
public boolean deleteByIsbn(long isbn) {
    if (search(isbn) == null) return false;
    root = deleteRecursive(root, isbn);
    return true;
}

// Sorted retrieval — In-order traversal O(n)
public List<Book> toList() {
    List<Book> books = new ArrayList<>();
```

```
    addBooksInOrder(root, books);
    return books;
}
```

Appendix C: BorrowHistoryStack Core Methods

```
// Push — O(1)
public void push(LoanRecord loanRecord) {
    top = new Node(loanRecord, top);
}

// Find active loan — O(n)
public LoanRecord findActiveLoanByIsbn(long isbn) {
    Node current = top;
    while (current != null) {
        LoanRecord lr = current.loanRecord;
        if (!lr.isReturned() && lr.getBook().getIsbn() == isbn)
            return lr;
        current = current.next;
    }
    return null;
}

// Convert to list — O(n), maintains LIFO order
public List<LoanRecord> toList() {
    List<LoanRecord> records = new ArrayList<>();
    Node current = top;
    while (current != null) {
        records.add(current.loanRecord);
        current = current.next;
    }
    return records;
}
```